

1. PostgreSQL sintaksa

1.1 Pretraživanje teksta

SIMILAR TO

znakovniNiz SIMILAR TO uzorak [ESCAPE specijalniZnak]
 specijalni znakovi:
 | (ili - jedna od dvije mogućnosti)
 * (ponavljanje prethodnog znaka/znakova 0 ili više puta)
 + (ponavljanje prethodnog znaka/znakova 1 ili više puta)
 () (grupiranje znaka/znakova u jednu logičku cjelinu)

Regularni izrazi

znakovniNiz ~ regularniIzraz
 ~ odgovara
 ! negacija (!~ ne odgovara)
 * neosjetljiv na velika i mala slova (!~*)

1.1.1. Pretraživanje teksta temeljem morfologije, semantike i sintakse jezika

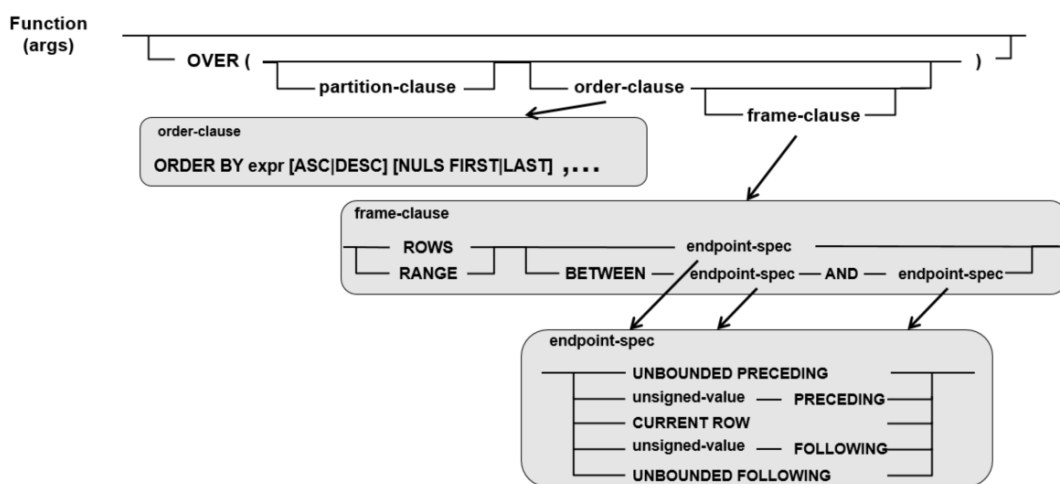
	Primjer
to_tsvector([config regconfig,] document text)	to_tsvector('english', 'The Fat Rats')
to_tsquery([config regconfig,] query text)	to_tsquery('english', 'The & Fat & Rats')
plainto_tsquery([config regconfig,] query text)	plainto_tsquery('english', 'The Fat Rats')
ts_rank([weights float4[],] vector tsvector, querytsquery [, normalization integer])	
ts_rank_cd([weights float4[],] vector tsvector, querytsquery [, normalization integer])	
ts_headline([config regconfig,] document text, querytsquery [, options text])	

FTS operator: @@
 TSVector @@ TSQuery
 TSQuery @@ TSVector
 TEXT|VARCHAR @@ TEXT|TSQuery

1.1.2. Približno pretraživanje teksta

soundex(text) returns text
levenshtein(text source, text target) returns int
metaphone(text source, int max_output_length) returns text
similarity(text, text)
show_trgm(text)
show_limit()
set_limit(real)
Operatori koji koriste funkciju similarity:
text % text
text <-> text

1.2. Particije-prozori-okviri



Neke od ugrađenih funkcija za rad s prozorima

row_number()	Redni broj n-torke u particiji, počinje od 1 za svaku particiju
rank ()	Rang n-torke u particiji, za jednake vrijednosti vraća jednak rang
dense_rank()	Jednako kao rank () ali bez "rupa" u slučaju jednako rangiranih n-torki
percent_rank()	relativni rang n-torke: (rang-1)/(ukupno n-torki -1)
lag(value any [,offset integer[,default any]])	Vraća vrijednost koja je izračunata za n-torku koja se nalazi u istoj particiji offset n-torki prije tekuće.
lead(value any [,offset integer [,default any]])	Jednako kao lag osim što n-torku traži offset n-torki nakon tekuće
first_value(value any)	Vraća vrijednost koja je izračunata za prvu n-torku u okviru
last_value(value any)	Vraća vrijednost koja je izračunata za zadnju n-torku u okviru
nth_value(value any, nth integer)	Vraća vrijednost koja je izračunata za n-torku koja je na nth-oj poziciji u okviru (počinje od 1).

1.3. Rekurzivni upiti

```
WITH RECURSIVE  
  R AS (SQL upit  
        UNION  
        rekurzivni SQL upit) //rekurzivni SQL upit uključuje R  
<SQL upit koji uključuje R (i druge relacije)>
```

1.4. Pivotiranje

crosstab (text sql)	Proizvodi "pivot tablicu" koja sadrži imena redaka i N imena stupaca. N (tip i broj vrijednosti) je određen upitom. SELECT * FROM crosstab('SELECT ...') AS pivotTable (prvaVrijUREdu tipPodatka, kat1 tipPodatka, kat2 tipPodatka,...);
crosstab (text srcSql, text categorySql)	Proizvodi "pivot tablicu". Stupci pivot tablice su specificirani drugim upitom (argumentom). SELECT * FROM crosstab('SELECT ...', 'SELECT ...') AS pivotTable (prvaVrijUREdu tipPodatka, kat1 tipPodatka, kat2 tipPodatka,...);

1.5. Objektno-relacijska proširenja - PostgreSQL

ARRAY

Operator	Opis
=, <>	usporedba ARRAY[1,2,3] = ARRAY[1,2,3]
@>	sadrži
&&	imaju li polja zajedničke elemente
	konkatenacija polja

Funkcija	Opis
array_length (polje,dim)	broj elemenata (za zadanu dimenziju <i>dim</i>)
array_lower (polje,dim)	donja granica (indeks) zadane dimenzije polja <i>dim</i>
array_upper (polje,dim)	gornja granica (indeks) zadane dimenzije polja <i>dim</i>
array_append (polje, el)	dodavanje elementa na kraj polja
array_prepend (el, polje)	dodavanje elementa na početak polja
array_remove (polje, el)	brisanje elemenata vrijednosti <i>el</i> iz jednodimenzionalnog polja
unnest (polje)	kreira jednu n-torku za svaki element polja
generate_subscripts (polje, dim [, desc])	generira n-torke koje odgovaraju indeksima elemenata polja (za zadanu dimenziju <i>dim</i>)
array_agg (izraz)	agregatna funkcija - kreira polje iz vrijednosti svake n-torke iz grupe

ROW konstruktor

ROW (literal, ...) npr.

```
INSERT INTO poduzece
VALUES ( 'INA',
        ROW ( 10000, 'Zagreb',
              ROW ('HR', 'Hrvatska')
            )
);
```

```
CREATE [ OR REPLACE ] FUNCTION name ( [ [ argmode ] [ argname ] argtype ] [, ... ] )
[ RETURNS rettype ]
{ LANGUAGE lang_name
| AS definition
}
```

argmode - npr. IN, OUT, INOUT

rettype - može uključivati SETOF ključnu riječ

definition - npr. SQL naredba: \$\$SELECT * FROM mjesto WHERE pbr = \$1;\$\$

```
CREATE DOMAIN domName [AS] dataType [ DEFAULT expression ] [ NOT NULL ] [ CHECK expression ]
```

```
CREATE TYPE name AS ( attribute_name data_type [, ... ] )
```

```
CREATE CAST (source_type AS target_type)
WITH FUNCTION function_name (argument_type [, ...])
```

```
CREATE TABLE table name (
{ column_name data_type [ DEFAULT default_expr ] [ column_constraint [ ... ] ]
| table_constraint
[ INHERITS ( parent_table [, ... ] ) ]
[WITH ( OIDS=TRUE)]
```

1.6. Temporalno proširenje - PostgreSQL

1.6.1. Tipovi podataka

	Primjer:
interval [fields] [(p)]	'P1-2-3T4:55:59'::interval ≡ 1 year 2 mons 3 days 04:55:59
daterange	'[01.10.2010, 1.10.2014)'
tsrange	'(12.11.2015 08:15:00, 12.11.2015 11:00:00]'
tstzrange	'[12.11.2015 08:15:00 CEST, 12.11.2015 11:00:00 CEST]'

1.6.2. Operatori i funkcije nad range tipovima

=	Equals
<>	not equal
<, <=	less then
>, >=	greater then
@>	sadrži raspon ili element (contains)
<@	sadržan je u
&&	preklapanje (overlap)
<<, >>	strogo lijevo/desno (strictly left/right)
&<, &>	ne prostire se desno/lijevo od (does not extend to the right/left of)
- -	susjedan (is adjacent to)
+	uniја
*	presjek
-	razlika

1.6.3. EXCLUDE ograničenje

```
ALTER TABLE tableName ADD CONSTRAINT constraintName  
EXCLUDE [ USING index_method ] ( exclude_element WITH operator [, ... ] )  
index_parameters [ WHERE ( predicate ) ]
```

Za period tip podatka EXCLUDE prihvaća
komutativne operatore: && i -|-
ne prihvaća nekomutativne : @>, <@, <<, >> , &<, &>

1.7. Odabrane PostgreSQL PostGIS funkcije

Spatial Relationships and Measurements	
float ST_Area (geometry <i>g1</i>);	Returns the area of the surface if it is a Polygon or MultiPolygon. For geometry, a 2D Cartesian area is determined with units specified by the SRID. For geography, area is determined on a curved surface with units in square meters.
boolean ST_Contains (geometry <i>geomA</i> , geometry <i>geomB</i>);	Returns true if and only if no points of B lie in the exterior of A, and at least one point of the interior of B lies in the interior of A.
boolean ST_ContainsProperly (geometry <i>geomA</i> , geometry <i>geomB</i>);	Returns true if B intersects the interior of A but not the boundary (or exterior). A does not contain properly itself, but does contain itself.
boolean ST_Crosses (geometry <i>g1</i> , geometry <i>g2</i>);	Returns TRUE if the supplied geometries have some, but not all, interior points in common.
boolean ST_Disjoint (geometry <i>A</i> , geometry <i>B</i>);	Returns TRUE if the Geometries do not "spatially intersect" - if they do not share any space together.
float ST_Distance (geometry <i>g1</i> , geometry <i>g2</i>);	For geometry type Returns the 2D Cartesian distance between two geometries in projected units (based on spatial ref). For geography type defaults to return minimum geodesic distance between two geographies in meters.
boolean ST_DWithin (geometry <i>g1</i> , geometry <i>g2</i> , double precision <i>distance_of_srid</i>);	Returns true if the geometries are within the specified distance of one another. For geometry units are in those of spatial reference.
boolean ST_Equals (geometry <i>A</i> , geometry <i>B</i>);	Returns true if the given geometries represent the same geometry.
boolean ST_Intersects (geometry <i>geomA</i> , geometry <i>geomB</i>);	Returns TRUE if the Geometries/Geography "spatially intersect in 2D" - (share any portion of space) and FALSE if they don't (they are Disjoint).
float ST_Length (geometry <i>a_2dlinestring</i>);	Returns the 2D length of the geometry if it is a LineString or MultiLineString. geometry are in units of spatial reference and geography are in meters (default spheroid)
boolean ST_Overlaps (geometry <i>A</i> , geometry <i>B</i>);	Returns TRUE if the Geometries share space, are of the same dimension, but are not completely contained by each other.
boolean ST_Touches (geometry <i>g1</i> , geometry <i>g2</i>);	Returns TRUE if the geometries have at least one point in common, but their interiors do not intersect.
boolean ST_Within (geometry <i>A</i> , geometry <i>B</i>);	Returns true if the geometry A is completely inside geometry B
Geometry Processing	
geometry ST_Buffer (geometry <i>g1</i> , float <i>radius_of_buffer</i>);	Returns a geometry covering all points within a given distance from the input geometry.
geometry ST_Centroid (geometry <i>g1</i>);	Computes the geometric center of a geometry, or equivalently, the center of mass of the geometry as a POINT.
geometry ST_ConvexHull (geometry <i>geomA</i>);	Computes the convex hull of a geometry. The convex hull is the smallest convex geometry that encloses all geometries in the input. In the general case the convex hull is a Polygon. The convex hull of two or more collinear points is a two-point LineString. The convex hull of one or more identical points is a Point. It is usually used with Multi* and GeometryCollections. It is not an aggregate function. To compute the convex hull of a set of geometries, use ST_Collect to aggregate them.
geometry ST_Difference (geometry <i>geomA</i> , geometry <i>geomB</i>);	Returns a geometry that represents that part of geometry A that does not intersect with geometry B.
geometry ST_Intersection (geometry <i>geomA</i> , geometry <i>geomB</i>);	Returns a geometry that represents the shared portion of geomA and geomB.
geometry ST_SymDifference (geometry <i>geomA</i> , geometry <i>geomB</i>);	Returns a geometry that represents the portions of A and B that do not intersect. It is called a symmetric difference because ST_SymDifference(A,B) = ST_SymDifference(B,A).
geometry ST_Union (geometry <i>g1</i> , geometry <i>g2</i>); geometry ST_Union (geometry[] <i>g1_array</i>);	Returns a geometry that represents the point set union of the Geometries.
geometry ST_Collect (geometry <i>g1</i> , geometry <i>g2</i>); geometry ST_Collect (geometry[] <i>g1_array</i>);	Creates a GeometryCollection or Multi* geometry from a set of geometries. The result is either a Multi* or a GeometryCollection, depending on whether the input geometries have the same or different types (homogeneous or heterogeneous). The input geometries are left unchanged within the collection.

2. Semantički web

2.1 RDF, RDFS, OWL

RDF	Instance relation	rdf:type
	Property	rdf:Property
RDFS	Subclass relation	rdfs:subClassOf
	Domain of a property	rdfs:domain
	Range of a property	rdfs:range
	Subproperty	rdfs:subPropertyOf
OWL – Concepts	Ontology concept	owl:Class
	Class equivalence	owl:equivalentClass
	Class disjointness	owl:disjointWith
	Class intersection	owl:intersectionOf
	Class union	owl:unionOf
	Class negation	owl:complementOf
OWL – Properties	Data property	owl:DatatypeProperty
	Object property	owl:ObjectProperty
	Property equivalence	owl:equivalentProperty
	Inverse property	owl:inverseOf
OWL – Individuals	Instance equivalence	owl:sameAs
	Instance difference	owl:differentFrom
OWL – Restrictions	Restriction	owl:Restriction
	Restriction property	owl:onProperty
	Existential quantifier	owl:someValuesFrom
	Universal quantifier	owl:allValuesFrom
	Specific value	owl:hasValue
	Minimum cardinality	owl:minCardinality
	Maximum cardinality	owl:maxCardinality
	Cardinality	owl:cardinality
OWL – Object Property Axioms	Inverse properties	owl:inverseOf
	Equivalent properties	owl:equivalentProperty
	Symmetric property	owl:SymmetricProperty
	Asymmetric property	owl:AsymmetricProperty
	Functional property	owl:FunctionalProperty
	Inverse functional property	owl:InverseFunctionalProperty
	Transitive property	owl:TransitiveProperty
	Reflexive property	owl:ReflexiveProperty
	Irreflexive property	owl:IrreflexiveProperty
	Disjoint properties	owl:propertyDisjointWith

2.1 SPARQL query syntax

```

Declare prefix shortcuts (optional) {
  PREFIX dbo: <...>
  PREFIX dbp: <...>
  ...
  SELECT/ASK/DESCRIBE/CONSTRUCT ...
  FROM <...> ← Define the dataset (optional)
  WHERE {
    ...
  } ← Query pattern
  ...
  GROUP BY ...
  HAVING ...
  ORDER BY ...
  LIMIT ...
  OFFSET ...
}
Query modifiers (optional)

```

2.1 Common namespaces and prefixes you can use in Turtle and SPARQL:

	Prefix	Namespace URL
XMLSchema	xsd	http://www.w3c.org/2001/XMLSchema#
RDF	rdf	http://www.w3.org/1999/02/22-rdf-syntax-ns#
RDFS	rdfs	http://www.w3.org/2000/01/rdf-schema#
OWL	owl	http://www.w3.org/2002/07/owl#
DBpedia resource	dbp	http://dbpedia.org/resource/
DBpedia ontology	dbo	http://dbpedia.org/ontology/
Schema.org	schema	http://schema.org/
Example	ex	http://www.example.org/#

```
// Few useful JS snippets for solving AdvDB HW
console.log("DATES");
// Date stuff:
var minDate = new Date("2008-10-04T07:14:00.000Z");
var maxDate = new Date("2018-10-11T01:38:00.000Z");
// Diff in days:
var diff = Math.round((maxDate-minDate)/(1000*60*60*24));
console.log(diff);
// Extract year:
console.log(maxDate.getFullYear());
console.log("\n\nROUNDING NUMBERS");
var pi = 3.14159;
console.log('π', pi);
pi = Math.round(pi * 1000)/1000; // to three decimals
console.log('π to three decimals', pi);
var br100 = 100.9999999;
br100 = Math.round(br100 * 1000)/1000; // to three decimals
console.log(100.9999999, 'ROUNDED to three decimals', br100);
console.log(100.9999999, 'ROUNDED and FORMATTED to three decimals ', br100.toFixed(3));
```

```
// Array stuff:
console.log("\n\nARRAYS");
var arr1 = [1, 2, 3];
var arr2 = [6, 5, 4, 3, 2, 1];
arr1 = arr1.concat(arr2);
console.log(arr1);
arr1.sort(); // sort CHANGES the array
console.log('default sort', arr1);
arr1.sort(function(a, b) {
    return b - a;
}); // sort CHANGES the array
console.log('custom sort (using compare function)', arr1);
var arrLastname = ["fff", "aaa", "ččč", "xxx", "zzz"];
arrLastname.sort(function(a, b) {
    return a.localeCompare(b);
}); // sort CHANGES the array
console.log('arrLastname', arrLastname)
// DISTINCT array:
var darr1 = arr1.filter((obj, idx, arr) => (
    arr.findIndex((o) => o === obj) === idx
));
console.log('distinct array', darr1);
// carfully, if you have objects:
var arrobj = [{id: 1, name: "A"}, {id: 1, name: "A"}];
var darr2 = arrobj.filter((obj, idx, arr) => (
    arr.findIndex((o) => o === obj) === idx
));
console.log('NOT distinct array', darr2);
// instead, eg. by ID:
var darr3 = arrobj.filter((obj, idx, arr) => (
    arr.findIndex((o) => o.id === obj.id) === idx
));
console.log('distinct array', darr3);
```

```
// Object stuff:
console.log("\n\nOBJECTS");
var obj = {};
console.log(obj);
obj.a = 'a-value';
obj['b'] = 'b-value';
console.log(obj);
Object.keys(obj).forEach(key => {
  console.log('object key:', key, 'object value:', obj[key]);
});
// Let's use it as a hash map:
// First, remove a and b:
delete obj.a;
delete obj.b;
arr1.forEach(function(elem) { // or you can use for(i = 0; i < arr1.length; ++i)
  if (obj[elem]) obj[elem]++;
  else obj[elem] = 1;
});
console.log(obj);
```

```
console.log("\n\nSTRING OPS");
var s = "A Woodchuck would chuck as much wood as a woodchuck could chuck if a woodchuck could chuck wood";
console.log('s.indexOf("wood")', s.indexOf('wood'));
console.log('s.indexOf("metal")', s.indexOf('metal'));
console.log('s.search("wood")', s.search('wood'));
console.log('s.search("wood")', s.search(/wood/));
console.log('s.search(/wOod/i)', s.search(/wOod/i));
console.log('s.search(/wOod/ig)', s.search(/wOod/ig));
console.log('s.match(/wOod/i)', s.match(/wOod/i));
console.log('s.match(/wOod/ig)', s.match(/wOod/ig));
console.log("\n Via variable:");
// via a variable:
var targets = ['wood', 'Chuck'];
targets.forEach(function(elem) { // or you can use for(i = 0; i < arr1.length; ++i)
  let re = new RegExp(elem, "gi");
  console.log(re, s.match(re));
});
// Be careful with regexs, might not work identical in mongo's js env
// eg try splitting string in words
console.log(s.match(/\S+/g));
```

Output:

```
DATES
3659
2018
```

ROUNDING NUMBERS

```
π 3.14159
π to three decimals 3.142
100.9999999 'ROUNDED to three decimals' 101
100.9999999 'ROUNDED and FORMATTED to three decimals' '101.000'
```

ARRAYS

```
[ 1, 2, 3, 6, 5, 4, 3, 2, 1 ]  
default sort [ 1, 1, 2, 2, 3, 3, 4, 5, 6 ]  
custom sort (using compare function) [ 6, 5, 4, 3, 3, 2, 2, 1, 1 ]  
arrLastname [ 'aaa', 'ččč', 'fff', 'xxx', 'zzz' ]  
distinct array [ 6, 5, 4, 3, 2, 1 ]  
NOT distinct array [ { id: 1, name: 'A' }, { id: 1, name: 'A' } ]  
distinct array [ { id: 1, name: 'A' } ]
```

OBJECTS

```
{ }  
{ a: 'a-value', b: 'b-value' }  
object key: a object value: a-value  
object key: b object value: b-value  
{ '1': 2, '2': 2, '3': 2, '4': 1, '5': 1, '6': 1 }
```

STRING OPS

```
s.indexOf("wood") 32  
s.indexOf("metal") -1  
s.search("wood") 32  
s.search("wood") 32  
s.search(/wOOd/i) 2  
s.search(/wOOd/ig) 2  
s.match(/wOOd/i) [ 'Wood',  
  index: 2,  
  input: 'A Woodchuck would chuck as much wood as a woodchuck could chuck if a woodchuck could chuck wood' ]  
s.match(/wOOd/ig) [ 'Wood', 'wood', 'wood', 'wood', 'wood' ]
```

Via variable:

```
/wood/gi [ 'Wood', 'wood', 'wood', 'wood', 'wood' ]  
/Chuck/gi [ 'chuck', 'chuck', 'chuck', 'chuck', 'chuck', 'chuck' ]  
[ 'A',  
  'Woodchuck',  
  'would',  
  'chuck',  
  'as',  
  'much',  
  'wood',  
  'as',  
  'a',  
  'woodchuck',  
  'could',  
  'chuck',  
  'if',  
  'a',  
  'woodchuck',  
  'could',  
  'chuck',  
  'wood' ]
```